Format Roadmap Framework — Ketentuan Penyusunan Roadmap Framework

> **Dokumen ini adalah KETENTUAN untuk AI.** Dipakai saat menyusun roadmap sebuah **framework** (Laravel, NestJS, React, Vue, Next.js, Express, Django, Spring Boot, Godot, Tailwind, Bootstrap, dsb.).
> **Hasil akhir:** satu file `Roadmap/<Kategori>/Roadmap <Framework>.md`.
> **Struktur umum** (Filosofi, Prinsip Desain, level, benang merah, checkpoint) **mengacu ke** `Format Roadmap.md` — dokumen ini hanya mengatur **perbedaan & kurikulum inti framework**.
> **Untuk tool/teknologi non-bahasa (Git, Docker, SQL, HTML/CSS): pakai** `Format Roadmap Tool & Teknologi.md`.

---

0. Kapan Dipakai & Beda dengan Roadmap Bahasa

Dipakai untuk framework/perpustakaan yang **dibangun di atas sebuah bahasa**. Contoh kategori:

- Frontend: React, Vue, Svelte, Next.js, Nuxt
- Backend: Laravel, Symfony, NestJS, Express, Django, Flask, Spring Boot, Gin
- Mobile/Desktop: Flutter, React Native, .NET MAUI, Electron
- Game: Godot, Unity, phaser
- CSS Framework: Tailwind, Bootstrap

	Roadmap Bahasa (`Format Roadmap.md`)	**Roadmap Framework (dokumen ini)**
Fokus	Menguasai bahasa & sintaks	Menguasai cara kerja framework + membangun aplikasi dengannya
Kurikulum inti	21 materi fundamental	**20 kurikulum framework** (Bagian 3)
Prasyarat	HTML/CSS, logika, tooling	**Bahasa dasarnya sudah dikuasai** (21 fundamental)
Mengulang fundamental?	Ya (itu materinya)	**Tidak** — hanya dijelaskan bagian yang diubah/di-kustom framework
Konsep khas	fitur bahasa	fitur framework (lifecycle, DI, ORM, dsb.)

> **Prinsip kunci:** framework mengasumsikan bahasa dasarnya sudah bisa. Roadmap framework **wajib** menyatakan prasyarat ini di bagian atas.

---

1. Prinsip Wajib Roadmap Framework

Selain 7 prinsip universal di `Format Roadmap.md` Bagian 1, roadmap framework menambahkan:

#	Prinsip Tambahan	Artinya
8	**Bahasa Dulu, Framework Kemudian**	Setiap materi framework dikaitkan ke konsep bahasa yang mendasarinya. Contoh: "middleware NestJS = implementasi konsep request lifecycle + higher-order function."
9	**"Apa yang Framework Gantikan"**	Jelaskan masalah yang tadinya manual di bahasa native, lalu bagaimana framework menyederhanakannya. Contoh: routing manual → resource route Laravel.
10	**Konvensi di atas Konfigurasi**	Tekankan konvensi framework (penamaan, struktur) supaya tidak "melawan" framework.
11	**Satu Aplikasi Nyata**	Bangun satu aplikasi sungguhan dengan framework (mis. sistem perpustakaan versi Laravel/NestJS/React).

---

2. Struktur Baku File Roadmap Framework

Struktur heading sama dengan `Format Roadmap.md` Bagian 2. Perbedaan (delta) ada di bawah.

2.1 Kepala file — ada 3 tambahan wajib

```
# 🗺️ Roadmap <Framework>: <Subjudul>

## Filosofi Roadmap Ini
> **"<kutipan>"** — <alasan>

### Prinsip Desain
- **Satu Aplikasi, Tumbuh Bersama**: <aplikasi apa, dari X → Y → Z>
- **Bahasa Dulu, Framework Kemudian**: <bahasa dasar & materi yang diasumsikan sudah dikuasai>
- **Security dari Hari Pertama**: <...>
- **Benang Merah Eksplisit**: ...
```

```
### Prasyarat Sebelum Memulai
```

Sebelum roadmap framework ini, pastikan sudah:
├── Menyelesaikan Roadmap <Bahasa> — khususnya materi 1-21
├── Menguasai
├── Terpasang <tool/package manager: Node/npm, PHP/Composer, Python/pip>
└── <pengetahuan dasar lain: HTTP, Git>

```
### Spesifikasi Aplikasi  ← TAMBAHAN
| Aspek | Keterangan |
|---|---|
| Nama Aplikasi | <mis. Perpustakaan Digital dengan Laravel> |
| Deskripsi | <1-2 kalimat> |
| Fitur Utama | <daftar fitur> |
| Tech Stack | <framework + database + tools> |
| Target Akhir | <aplikasi terdeploy> |
```

2.2 Gambaran besar

Sama seperti `Format Roadmap.md`. Alur level khas framework:

```
Level 1: "Fondasi" — filosofi, instalasi, struktur folder, konvensi
    ↓
Level 2: + Fitur inti (routing, controller/handler, view/component)
    ↓
Level 3: + Data (ORM/ODM, migration, relasi)
    ↓
Level 4: + Auth, validasi, otorisasi
    ↓
Level 5: + Fitur lanjutan (API, queue/event, real-time, caching)
    ↓
Level 6: + Testing, keamanan menyeluruh, performa
    ↓
Level 7: + Build & deployment
```

2.3 Tiap level — tambah baris "Fitur Framework"

```
## 🟢 LEVEL N: <TEMA> (Minggu a-b)

> **Tema**: _<...>_
> **Benang Merah**: <...>
```

```
> **Output**: <...>
> **Fitur Framework yang Dipelajari**: <routing, middleware, ORM, dsb.>

---

### A. <Nama Bagian>

> 💡 **Mengapa dimulai di sini?** <alasan>
```

Benang Merah Bagian A:
<...> → <...> → <...>

```
1. `[[<Nomor>. <Judul Materi Framework>]]`

    - <poin>
    - **Padanan di bahasa native**: <bagaimana ini dilakukan tanpa framework>
← dorong pembaca paham "yang digantikan framework"
    - _Langkah konkret_: <aksi: jalankan CLI, buat file, uji endpoint>
```

2.4 Checkpoint — tambah blok KUALITAS

```
### 🚦 Checkpoint Level N
```

✅ Checklist sebelum lanjut ke Level N+1:

PEMAHAMAN:
├── Bisa jelaskan
└── Bisa jelaskan padanan native-nya

FITUR:
├── <endpoint/halaman yang harus jalan>
└──

KUALITAS:
├── <migration/seed jalan>
├── <validasi & error handling aktif>
└──

Git: <conventional commit, mis. feat: implement resource route for books>

3. Kurikulum Inti Framework (WAJIB tercakup)

Ini pengganti "21 materi fundamental". Roadmap framework HARUS membahas ke-20 poin ini (boleh digabung/dipecah, boleh beda urutan selama dependency benar).

No	Materi Framework	Yang harus dibahas
1	**Filosofi & Pola Arsitektur**	Masalah yang dipecahkan framework; prinsip (convention over configuration, MVC/MVVM/component-based); trade-off.
2	**Instalasi & Setup Project**	CLI/scaffold, package manager, versi, menjalankan dev server/build.
3	**Struktur Direktori & Konvensi**	Fungsi tiap folder & file; naming convention framework.
4	**Konfigurasi & Environment**	File config, `.env`, environment variable, config cache, variabel per-environment.
5	**Routing**	Route dasar, parameter, group, named route, resource/prefix, wildcard.
6	**Controller / Handler / Component**	Peran, dependency injection, request/response object, pemisahan tanggung jawab.
7	**View / Rendering / Komponen UI**	Template engine, layout & partial, komponen, data binding, props/slot.
8	**Model & ORM/ODM**	Model, query builder, relasi, migration, seeder/factory.
9	**Validasi Input**	Validasi request, pesan error, form request/schema/DTO.
10	**Request Lifecycle & Middleware**	Alur request dari masuk sampai respons; middleware/hook/pipe/guard/interceptor.
11	**Autentikasi**	Register/login, session/token/JWT, logout, remember me.
12	**Otorisasi**	Role, policy/guard, hak akses per-resource.
13	**State / Service Layer**	Frontend: state global, props, store, reactivity. Backend: service/repository layer.
14	**Komunikasi API**	Frontend: konsumsi API (fetch/axios), loading & error state. Backend: menyediakan REST/GraphQL, serialisasi resource.
15	**Penanganan Error & Logging**	Error handler global, format error konsisten, logging & monitoring.

No	Materi Framework	Yang harus dibahas
16	**Keamanan**	CSRF, XSS, SQL Injection, CORS, sanitasi, rate limiting, hashing password.
17	**Testing**	Unit, feature/integration, e2e; mocking, factory, database test.
18	**Caching & Performa**	Cache, eager loading vs N+1, lazy loading, bundle size, optimasi query.
19	**Build & Deployment**	Build tool, environment production, deploy ke hosting/VPS/cloud, CI/CD.
20	**Design Pattern & Ekosistem**	DI, service/repository, action pattern, SOLID; package/plugin populer & komunitas.

> **Catatan adaptasi:** untuk framework frontend, poin ORM/migration diganti state management & routing client; untuk framework game, poin ORM diganti scene/node system. Ambil yang relevan dan jelaskan yang tidak berlaku.

---

4. Fitur Khas per Framework (wajib dibahas)

Setiap framework punya fitur pembeda. Masukkan sebagai bagian tersendiri di level yang tepat.

- **Laravel**: Eloquent ORM, Blade, Artisan CLI, migration & seeder, middleware, service container & DI, Form Request validation, model relationship, queue & job, event & listener, Sanctum/Passport, Livewire, Filament, task scheduling.
- **NestJS**: module, decorator, dependency injection (provider), controller & routing, guard, interceptor, pipe, exception filter, DTO + class-validator, ConfigModule, Prisma/TypeORM, terminologi microservices, Swagger.
- **React**: JSX, komponen & props, `useState` / `useEffect` & hooks lain, lifting state, context, memoization (`useMemo` / `useCallback`), reconciliation & virtual DOM, client-side routing, form handling, data fetching.
- **Vue**: reactivity (`ref` / `reactive`), directive (`v-if` / `v-for` / `v-model`), SFC, Composition API, computed & watch, lifecycle hooks, Vue Router, Pinia.
- **Next.js / Nuxt**: file-based routing, SSR/SSG/ISR/CSR, data fetching (`getServerSideProps` / `useFetch`), API routes, image & performance optimization, server actions.
- **Express**: middleware chain, routing, `req` / `res` , error-handling middleware, templating, integrasi DB & ORM.
- **Django / Flask**: ORM Django, admin panel, URLconf, template, forms; Flask: blueprint, extension, request context.

- **Spring Boot**: stereotype annotation, dependency injection, `@RestController`, Spring Data JPA, Spring Security, Actuator, profiler.
- **Flutter**: widget tree, stateful vs stateless, layout widget, state management (Provider/Riverpod/Bloc), navigasi, platform channel.
- **Godot**: scene & node, GDScript, signal, lifecycle (`_ready` / `_process`), physics, animasi, export.
- **Tailwind CSS**: utility-first, variant responsif, `@apply`, konfigurasi & purge, dark mode.
- **Bootstrap**: grid system, komponen, utility, kustomisasi Sass.

---

5. Contoh Alur Level

Framework backend (contoh: Laravel)

```
Level 1: Fondasi — filosofi, instalasi, struktur, artisan, routing dasar
Level 2: Controller, Blade, request/response, validasi
Level 3: Eloquent, migration, relasi, seeder
Level 4: Auth (Breeze/Manual), policy, middleware
Level 5: API (resource), queue, event, caching
Level 6: Testing, keamanan, performa
Level 7: Deployment
```

Framework frontend (contoh: React)

```
Level 1: Fondasi — filosofi, setup (Vite), JSX, komponen & props
Level 2: State & hooks, event, form, conditional rendering
Level 3: Routing, data fetching, loading/error state
Level 4: State management global, context, memoisasi
Level 5: Testing, aksesibilitas, performa
Level 6: Build & deployment
```

---

6. Aturan Penomoran, Link, dan Konsistensi

Sama seperti `Format Roadmap.md` Bagian 6, dengan penekanan:

1. Nomor materi **berurutan global** lintas level.
2. Setiap poin **wajib** memuat "Padanan di bahasa native" agar pembaca melihat apa yang digantikan framework.

3. Link materi: `` `[[<Nomor>. <Judul> – <Subjudul>]]` `` (tanpa path, aman karena nama unik).
4. Setiap nomor harus punya file materi (dibuat memakai `Format Penjelasan Materi.md`).
5. Sebutkan **versi framework** yang dipakai (mis. Laravel 11, NestJS 10, React 19) supaya materi tidak basi.

---

7. Template Roadmap Framework (copy-paste)

```
# 🗺️ Roadmap <Framework>: <Subjudul>

## Filosofi Roadmap Ini

> **"<kutipan>"** – <alasan>

### Prinsip Desain

- **Satu Aplikasi, Tumbuh Bersama**: <...>
- **Bahasa Dulu, Framework Kemudian**: <bahasa dasar & materi asumsi>
- **Security dari Hari Pertama**: <...>
- **Benang Merah Eksplisit**: setiap langkah terhubung ke langkah sebelum dan sesudahnya

### Prasyarat Sebelum Memulai
```

Sebelum roadmap framework ini, pastikan sudah:
├── Menyelesaikan Roadmap <Bahasa> — materi 1-21
├── Menguasai
└── Terpasang <tool/package manager>

```
### Spesifikasi Aplikasi

| Aspek | Keterangan |
|---|---|
| Nama Aplikasi | <...> |
| Deskripsi | <...> |
| Fitur Utama | <...> |
| Tech Stack | <Framework> + <DB> + <tools> |
| Target Akhir | <...> |
```

```
---

## 📋 Gambaran Besar — Apa yang Akan Dibangun
```

Level 1: ...
↓
Level N: ...

```
---

## 🟢 LEVEL 1: <TEMA> (Minggu 1–x)

> **Tema**: _<...>_
> **Benang Merah**: <...>
> **Output**: <...>
> **Fitur Framework yang Dipelajari**: <...>

---

### A. <Bagian>

> 💡 **Mengapa dimulai di sini?** <...>
```

Benang Merah Bagian A:
<...> → <...> → <...>

```
1. `[[1. <Judul> — <Subjudul>]]`

    - <poin>
    - **Padanan di bahasa native**: <...>
    - _Langkah konkret_: <aksi>

---

### 🏁 Checkpoint Level 1
```

✅ Checklist sebelum lanjut ke Level 2:

PEMAHAMAN:
├── <...>

FITUR:
├── <...>

KUALITAS:
├── <...>

Git: feat: <...>

---

8. Checklist Kualitas Roadmap Framework

- [ ] Ada **Prasyarat** yang menunjuk `[[Roadmap <Bahasa>]]`.
- [ ] Ada **Spesifikasi Aplikasi** yang akan dibangun.
- [ ] Semua 20 poin **Kurikulum Inti Framework** tercakup (atau dijelaskan kenapa tidak berlaku).
- [ ] **Fitur khas framework** dibahas.
- [ ] Setiap poin punya **"Padanan di bahasa native"** (apa yang digantikan framework).
- [ ] Setiap level punya Tema, Benang Merah, Output, **Fitur Framework yang Dipelajari**.
- [ ] Checkpoint memuat blok **KUALITAS** (bukan hanya pemahaman & fitur).
- [ ] Versi framework disebutkan.
- [ ] Satu aplikasi nyata tumbuh dari level pertama sampai deploy.

File terkait:

- `Format Roadmap.md` — struktur umum & roadmap bahasa.
- `Format Roadmap Tool & Teknologi.md` — roadmap tool/teknologi non-bahasa.
- `Format Penjelasan Materi.md` — menulis materi tiap poin.
- `Format Pembuatan Project.md` — menyusun roadmap project.